

Estimating worst-case/average-case complexity of algorithms via linear and binary search:

Name

Soumayan Bej

Source Code

```
1 import time
2 import matplotlib.pyplot as plt
3
4 def linear(a, x):
5     for i in range(len(a)):
6         if a[i] == x:
7             return i
8
9 def test_linear(a, x):
10     t = time.perf_counter()
11     for _ in range(100):
12         linear(a, x)
13     return (time.perf_counter() - t) / 100
14
15
16 n = range(1000, 10001, 1000)
17 linear_best, linear_avg, linear_worst = [], [], []
18
19 for size in n:
20     a = list(range(size))
21
22     linear_best.append(test_linear(a, a[0]))
23     linear_avg.append(test_linear(a, a[size // 2]))
24     linear_worst.append(test_linear(a, -1))
25
26
27 def binary(a, x):
28     l, r = 0, len(a) - 1
29     while l <= r:
30         m = (l + r) // 2
31         if a[m] == x:
32             return m
33         if a[m] < x:
34             l = m + 1
35         else:
36             r = m - 1
37
38 def test_binary(a, x):
39     t = time.perf_counter()
40     for _ in range(100):
41         binary(a, x)
42     return (time.perf_counter() - t) / 100
43
```

```
44 binary_best, binary_avg, binary_worst = [], [], []
45
46
47 for size in n:
48     a = list(range(size))
49
50     binary_best.append(test_binary(a, a[size // 2]))
51     binary_avg.append(test_binary(a, a[size // 4]))
52     binary_worst.append(test_binary(a, -1))
53
54
55 plt.figure()
56 plt.plot(n, linear_best, label="Best Case")
57 plt.plot(n, linear_avg, label="Average Case")
58 plt.plot(n, linear_worst, label="Worst Case")
59 plt.xlabel("Input Size (n)")
60 plt.ylabel("Execution Time (seconds)")
61 plt.title("Linear Search - Best, Average and Worst Case")
62 plt.legend()
63 plt.grid()
64 plt.show()
65
66
67 plt.figure()
68 plt.plot(n, binary_best, label="Best Case")
69 plt.plot(n, binary_avg, label="Average Case")
70 plt.plot(n, binary_worst, label="Worst Case")
71 plt.xlabel("Input Size (n)")
72 plt.ylabel("Execution Time (seconds)")
73 plt.title("Binary Search - Best, Average and Worst Case")
74 plt.legend()
75 plt.grid()
76 plt.show()
```

Graphs

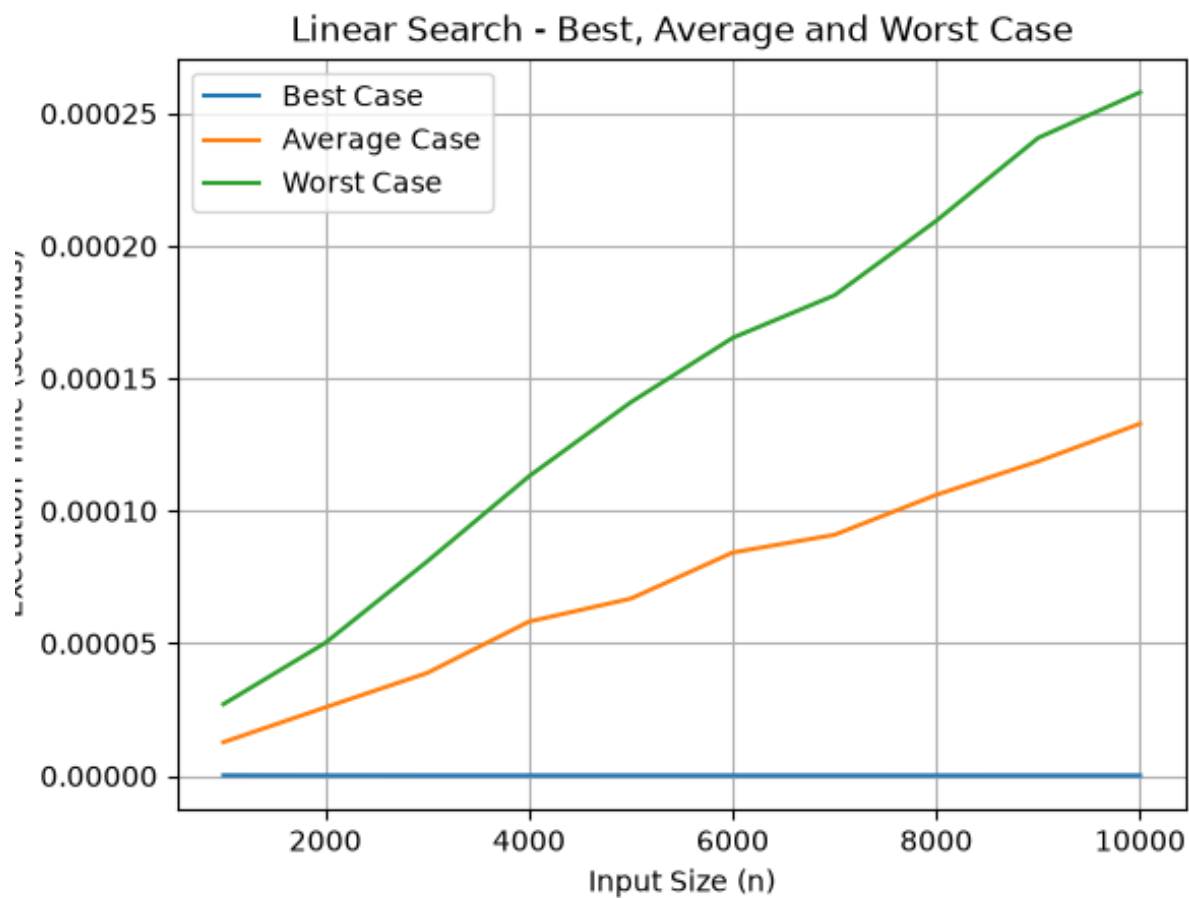


Figure 1: Linear Search- Best, Average, Worst case

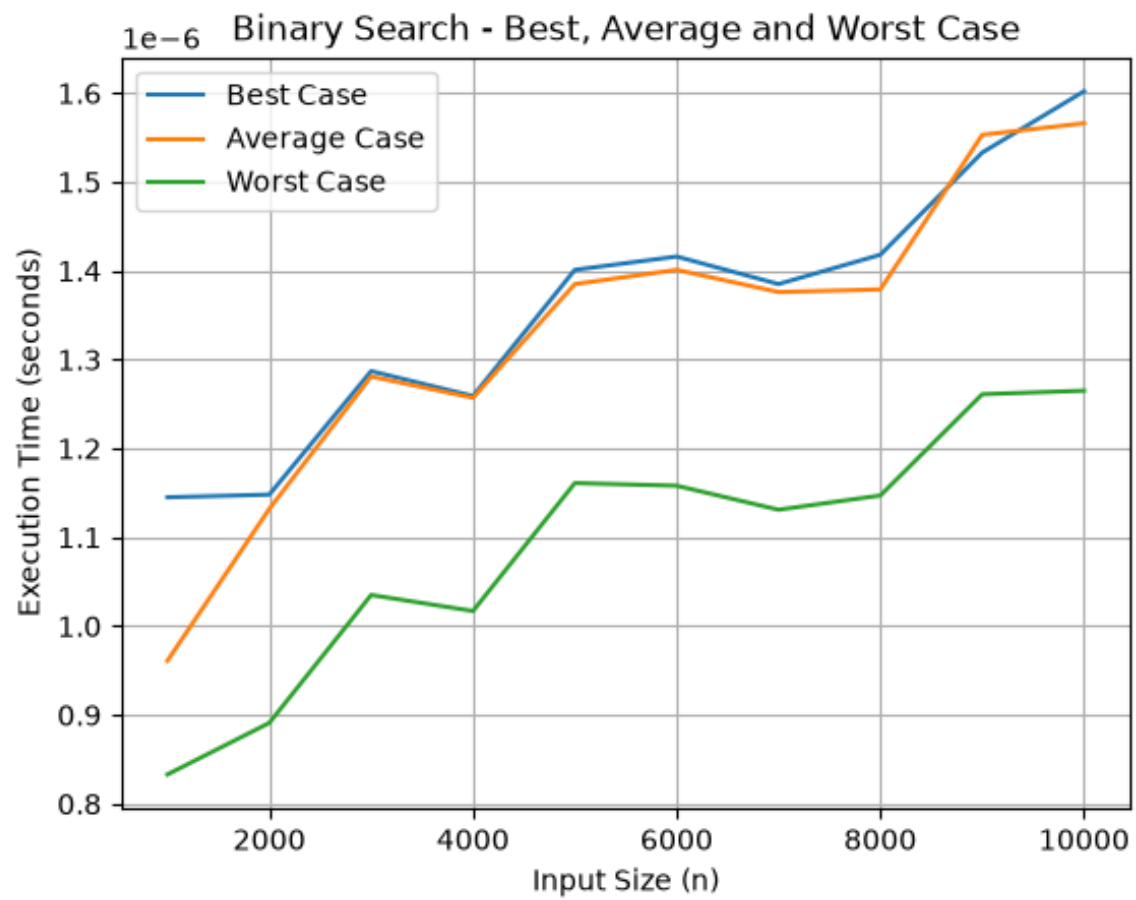


Figure 2: Binary Search- Best, Average, Worst Case